❷ Includes HSTS middleware if it is not development

Having configured your application for HTTPS, you have ensured communication is private and removed the prospect of *man-in-the-middle attacks*, when a bad actor intercepts web traffic and steals data or modifies content. The rest of this chapter will explore other common threats, as defined by the OWASP. You will understand the nature of vulnerabilities and how attacks can be performed. Once you appreciate the mechanics of a particular threat, you will learn how to mitigate it. You'll begin by looking at the area where the most common threats were found, according to the current 2021 OWASP Top Ten (https://owasp.org/www-project-top-ten/), which are grouped in a common category of broken access control.

13.2 Broken access control

An application's access control policies should be designed to restrict the operations a user is able to perform and the data they are able to access. This is known as the *least privilege principle*. The most common risk of broken access control is unauthorized access to, modification of, destruction of, or theft of data. In this section, we will look at some of the ways access control can become broken and how to prevent it. We will examine some best practices for applying and managing security policies first. Then we will look at the dangers cross-site forgery requests present and the mechanism Razor Pages provides to help protect against such attacks.

13.2.1 Incorrect security policies

Incorrect security policies is a broad category that covers missing, out-of-date, or incorrectly applied attempts to restrict access to unauthorized users (see chapter 10 for a review of the mechanics of authorization). There isn't a specific framework-based solution to this group of vulnerabilities, but there are some best practices you should try to apply as part of application development.

The first piece of guidance is to always implement your authorization policies on the server. As discussed a number of times previously, any client-side code is vulnerable to being amended or bypassed by any user who has minimal HTML or JavaScript knowledge.

At the time of writing, there is a recent case in which a news reporter discovered that the Social Security numbers of teachers were included in the HTML source code of web pages